

VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT) REPORT

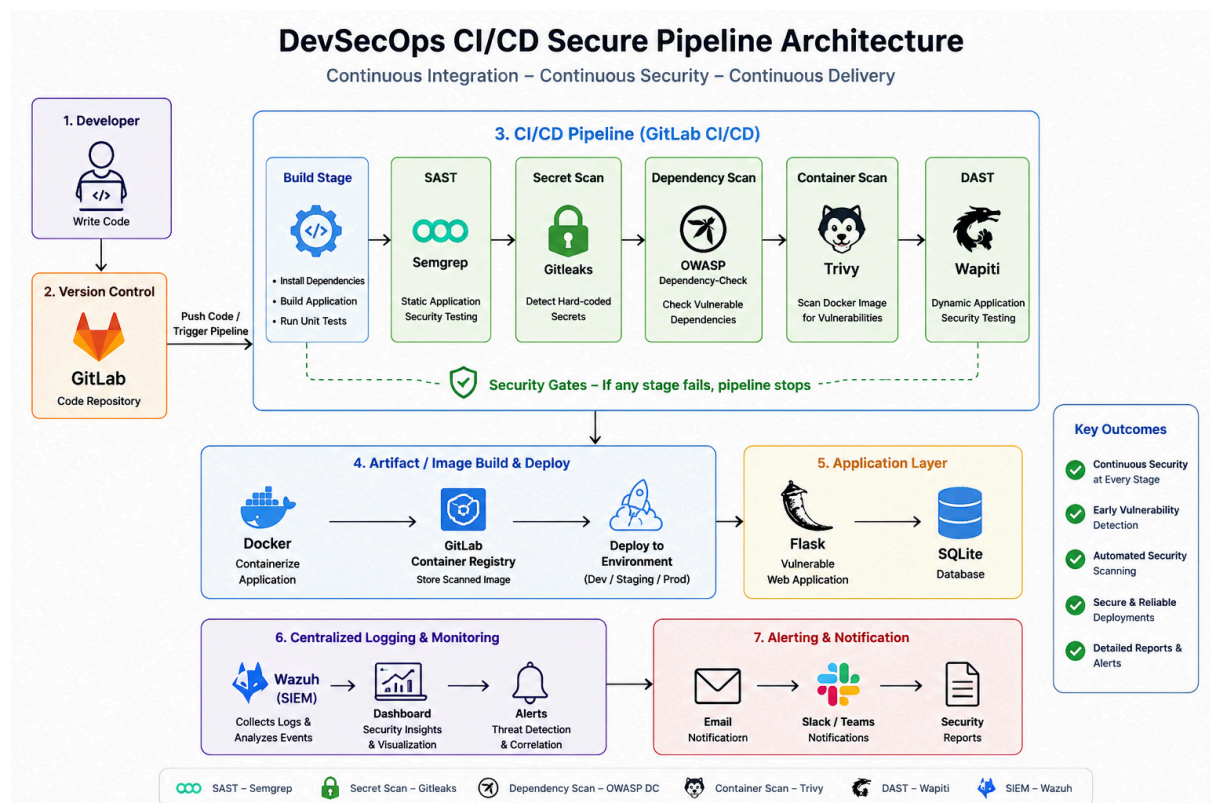
DevSecOps Secure CI/CD Pipeline Project

1. Executive Summary

This report presents the results of a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) conducted on a deliberately vulnerable Flask-based web application integrated within a DevSecOps CI/CD pipeline. The objective of this assessment was to identify, analyze, and validate security vulnerabilities using both automated and manual testing techniques.

The assessment identified multiple vulnerabilities across different severity levels, including critical issues such as SQL Injection and Hardcoded Secrets, as well as high-risk vulnerabilities like Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), and Broken Authentication. Additionally, medium-risk issues such as missing security headers and weak cryptographic practices were observed.

All findings have been mapped to OWASP Top 10 (2021), CWE standards, and evaluated using CVSS v3.1 scoring to determine their severity and business impact.



2. Scope of Assessment

In Scope:

- Flask-based Web Application
- GitLab CI/CD Pipeline
- Source Code (SAST)
- Secret Detection
- Dependency Analysis
- Dynamic Application Security Testing (DAST)

3. Methodology

3.1 Automated Testing

- Static Application Security Testing (SAST) using Semgrep
- Secret Scanning using Gitleaks
- Dynamic Application Security Testing (DAST) using Wapiti
- Dependency Analysis

3.2 Manual Testing

- SQL Injection exploitation
- Cross-Site Scripting payload execution
- IDOR enumeration
- Authentication bypass validation

3.3 Standards Referenced

- OWASP Top 10 (2021)
- CVSS v3.1

4. Tools Used

- Flask – to create a test web application
- GitLab CI/CD – for automation
- Semgrep – for static code analysis
- Gitleaks – for detecting exposed secrets
- Wapiti – for dynamic security testing
- SQLite – for database
- Docker – for containerization

5. Vulnerability Findings

5.1 SQL Injection

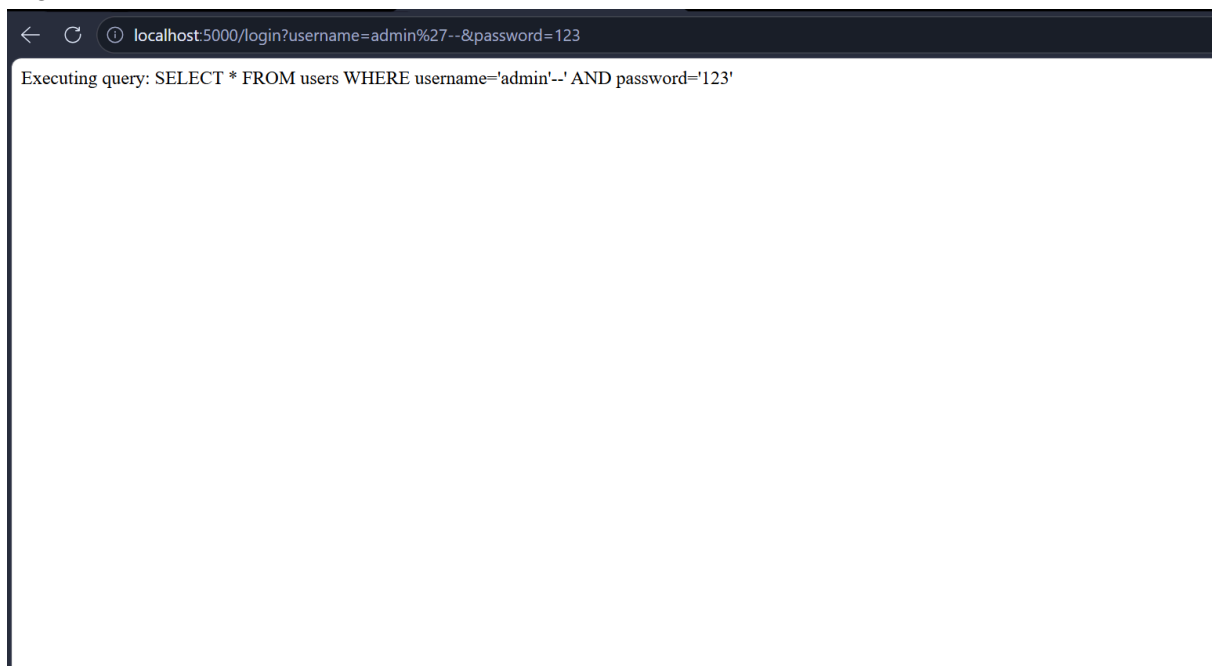
- **Severity:** Critical
- **CVSS Score:** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
- **OWASP:** A03:2021 – Injection
- **CWE:** CWE-89

Description

The application dynamically constructs SQL queries using unsanitized user input. This allows attackers to manipulate query logic and execute arbitrary SQL commands.

Proof of Concept

/login?username=admin'--&password=123



```
Gowry12 / devsecops-secure-pipeline / Jobs / #13862323892

Log timestamps in UTC. Search visible log output

49 05:05:54 // blocking //
50 05:05:54 User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and
51 05:05:54 therefore protected information could be leaked. Instead, use django's QuerySets, which are built
52 05:05:54 with query parameterization and therefore not vulnerable to sql injection. For example, you could
53 05:05:54 use `Entry.objects.filter(date=2006)`.
54 05:05:54 Details: https://sq.run/gx7y
55 05:05:54
56 05:05:54 38| username = request.args.get('username')
57 05:05:54 39| password = request.args.get('password')
58 05:05:54 40|
59 05:05:54 41| conn = sqlite3.connect('users.db')
60 05:05:54 42| cursor = conn.cursor()
61 05:05:54 43|
62 05:05:54 44| query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
63 05:05:54 45|
64 05:05:54 46| try:
65 05:05:54 47|     cursor.execute(query)
66 05:05:54 [hid 3 additional lines, adjust with --max-lines-per-finding]
67 05:05:54 39| password = request.args.get('password')
68 05:05:54 40|
69 05:05:54 41| conn = sqlite3.connect('users.db')
70 05:05:54 42| cursor = conn.cursor()
71 05:05:54 43|
72 05:05:54 44| query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
73 05:05:54 45|
74 05:05:54 46| try:
75 05:05:54 47|     cursor.execute(query)
76 05:05:54 48|     result = cursor.fetchone()
77 05:05:54 [hid 2 additional lines, adjust with --max-lines-per-finding]
```

Impact

- Authentication bypass
- Full database compromise
- Data exfiltration
- Privilege escalation

Recommendation

- Implement parameterized queries using prepared statements
- Avoid string concatenation in SQL queries
- Use ORM frameworks like SQLAlchemy
- Apply strict input validation and database access controls

5.2 Cross-Site Scripting (XSS)

- **Severity:** High
- **CVSS Score:** 8.2 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N)
- **OWASP:** A03:2021 – Injection

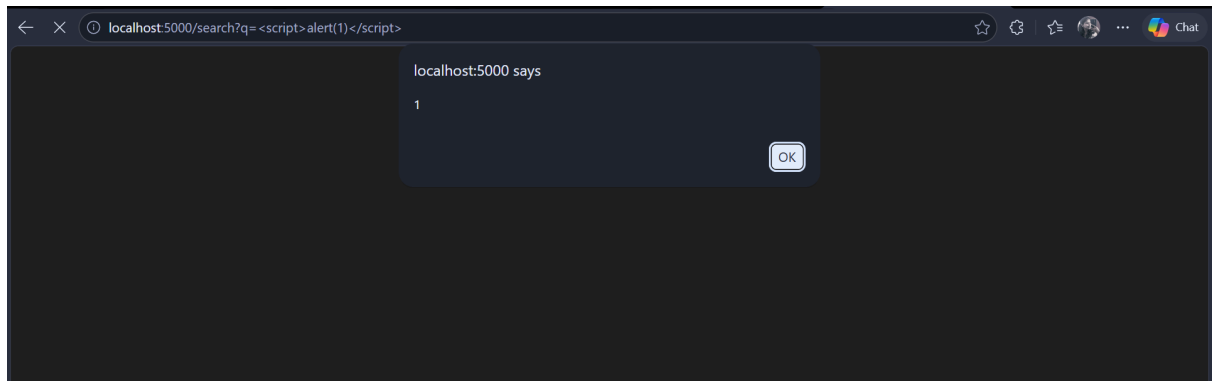
- **CWE:** CWE-79

Description

User input is reflected in the response without proper output encoding, enabling execution of malicious scripts in the browser.

Proof of Concept

/search?q=<script>alert(1)</script>



```
Log timestamps in UTC. Search visible log output

114 05:05:54 ORM will be a better option.
115 05:05:54 Details: https://sg.run/2b1l
116 05:05:54
117 05:05:54 47| cursor.execute(query)
118 05:05:54
119 05:05:54 )) python.flask.security.audit.directly-returned-format-string.directly-returned-form
at-string
120 05:05:54 (( Blocking ))
121 05:05:54 Detected Flask route directly returning a formatted string. This is subject to
cross-site scripting
122 05:05:54 if user input can reach the string. Consider using the template engine instead
and rendering pages
123 05:05:54 with 'render_template()'.
124 05:05:54 Details: https://sg.run/Zv6o
125 05:05:54
126 05:05:54 61| return f"Search: {q}"
127 05:05:54
128 05:05:54 )) python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
129 05:05:54 (( Blocking ))
130 05:05:54 Running flask app with host 0.0.0.0 could expose the server publicly.
131 05:05:54 Details: https://sg.run/elby
132 05:05:54
133 05:05:54 76| app.run(host='0.0.0.0', port=5000)
134 05:05:54
135 05:05:54 app/venv/lib/python3.12/site-packages/click/_termui_impl.py
136 05:05:54 ))) python.lang.compatibility.python36.python36-compatibility-Popen1
137 05:05:54 (( Blocking ))
138 05:05:54 the 'errors' argument to Popen is only available on Python 3.6+
139 05:05:54 Details: https://sg.run/weBP
140 05:05:54
141 05:05:54 454| c = subprocess.Popen(
142 05:05:54 455| [str(cmd_path)] + cmd_params,
143 05:05:54 456| shell=False,
```

SAST

```
Gowry12 / devsecops-secure-pipeline / Jobs / #13862323901

Log timestamps in UTC. Search visible log output

296 05:09:13      --  --  --  --  --
297 05:09:13      / / \ \ \_ _ _ _ _ ( ) | ( ) _ _ _ /
298 05:09:13      \ \ / \ / _ ' | ' \ | | _ | | _ \
299 05:09:13      \ / \ / ( | | | ) | | | | _ _ ) |
300 05:09:13      \ / \ \_ _ | _ _ / | \ _ _ | _ _ _ /
301 05:09:13      | _ |
302 05:09:13  Wapiti 3.2.3 (wapiti-scanner.github.io)
303 05:09:13  [*] Saving scan state, please wait...
304 05:09:13  [*] Launching module file
305 05:09:13  [*] Launching module ssl
306 05:09:13  sslscan is not installed or not found in PATH, module will be skipped
307 05:09:13  [*] Launching module cookieflags
308 05:09:13  [*] Launching module xss
309 05:09:13  [*] Launching module csp
310 05:09:13  127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
311 05:09:13  CSP is not set
312 05:09:13  [*] Launching module exec
313 05:09:13  [*] Launching module upload
314 05:09:13  [*] Launching module redirect
315 05:09:13  [*] Launching module sql
316 05:09:13  127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
317 05:09:13  [*] Launching module http_headers
318 05:09:13  127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
319 05:09:13  Checking X-Frame-Options:
320 05:09:13  X-Frame-Options is not set
321 05:09:13  Checking X-Content-Type-Options:
322 05:09:13  X-Content-Type-Options is not set
323 05:09:13  [*] Launching module ssrf
324 05:09:16  [*] Launching module permanentxss
325 05:09:16  127.0.0.1 - - [10/Apr/2026 05:09:16] "GET / HTTP/1.1" 200 -
326 05:09:16  [*] Generating report...
327 05:09:16  A report has been generated in the file report.txt
328 05:09:18  Uploading artifacts for successful job
```

DAST

Impact

- Session hijacking
- Credential theft
- Client-side attacks

Recommendation

- Enable output encoding (HTML escaping)
- Use Jinja2 auto-escaping
- Implement Content Security Policy (CSP)

- Validate and sanitize user input

5.3 Insecure Direct Object Reference (IDOR)

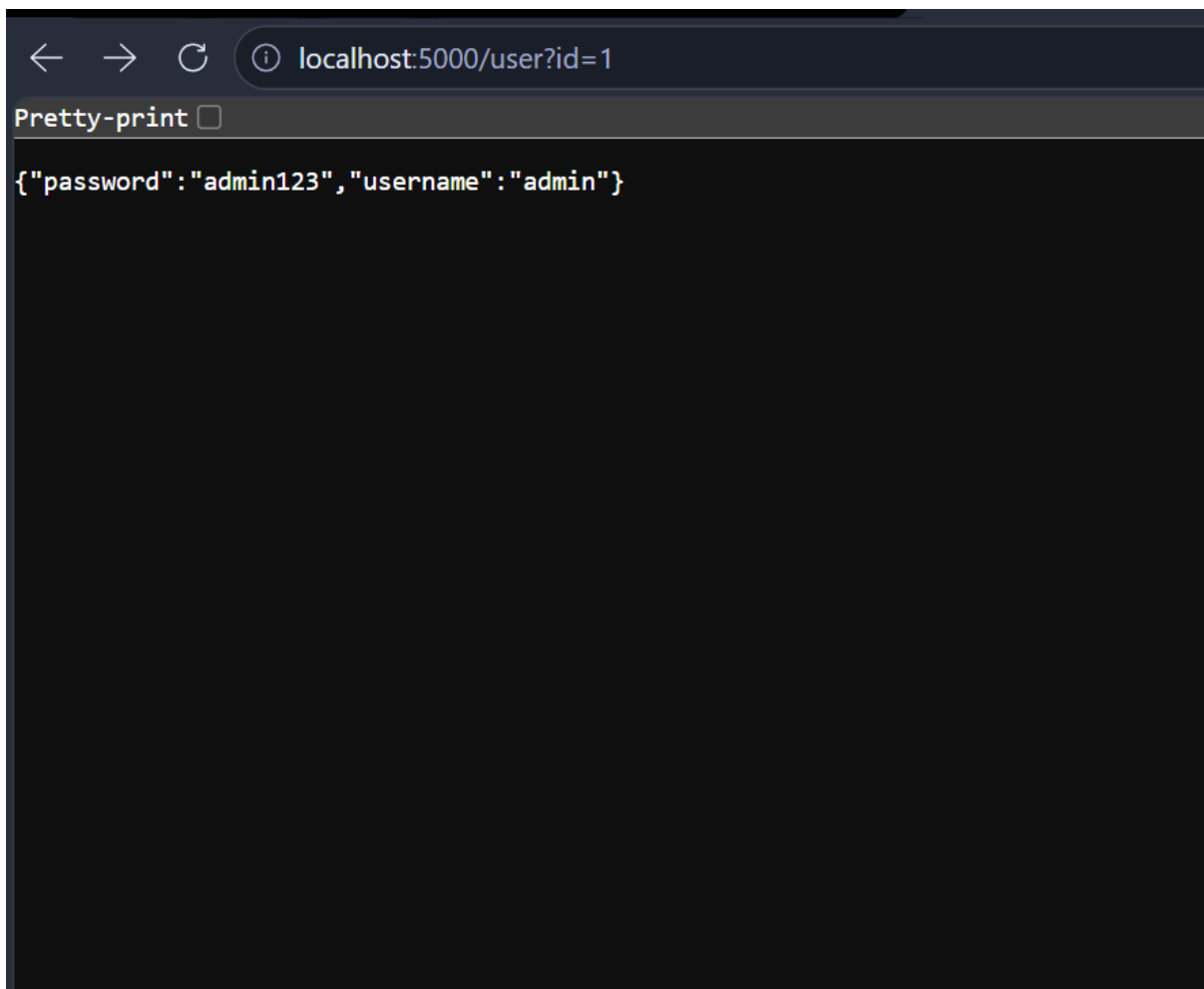
- **Severity:** High
- **CVSS Score:** 8.0 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
- **OWASP:** A01:2021 – Broken Access Control
- **CWE:** CWE-639

Description

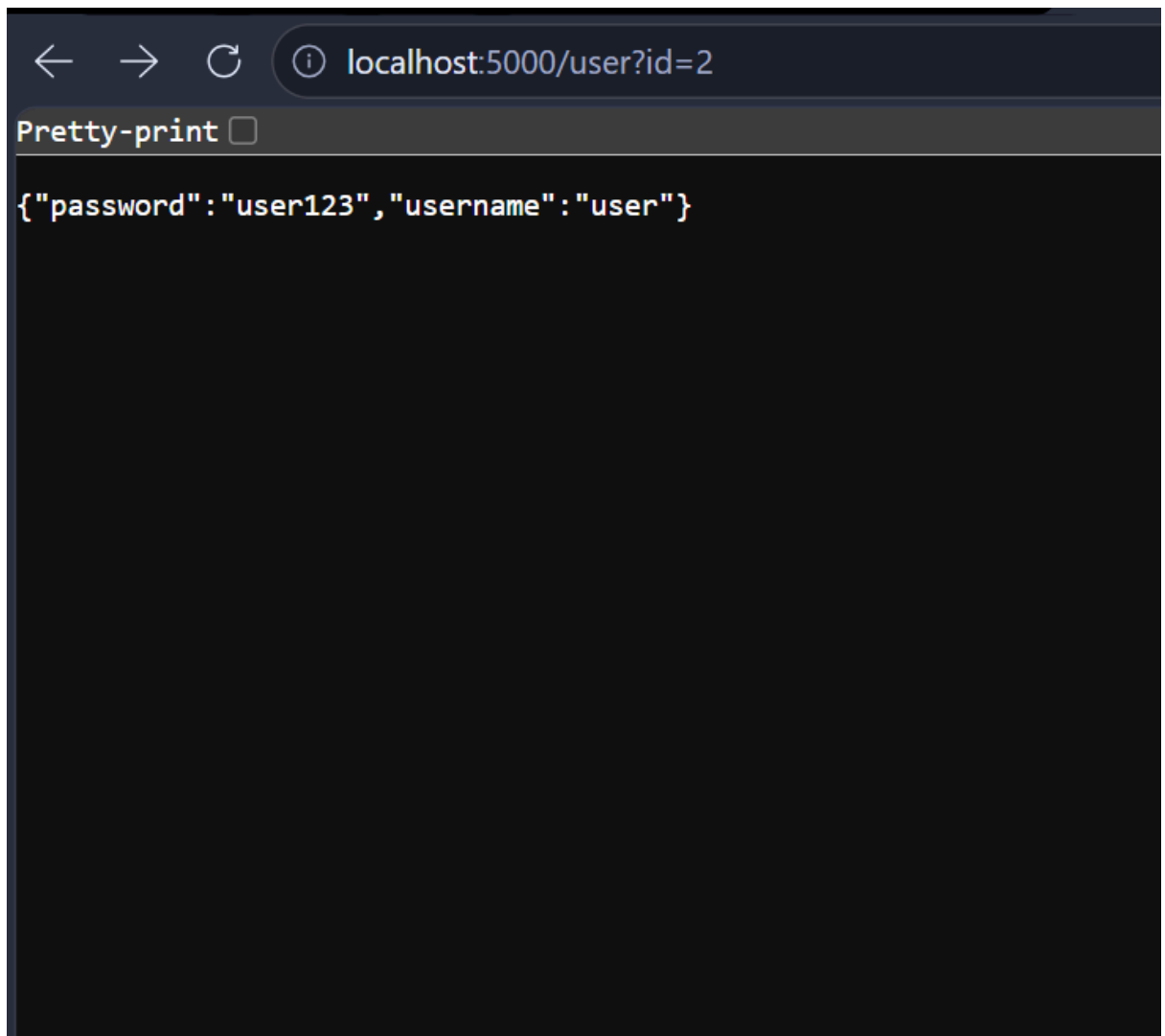
The application exposes user data based solely on user-supplied identifiers without enforcing authorization checks.

Proof of Concept

/user?id=1



/user?id=2



Impact

- Unauthorized data access
- Data leakage
- Privacy violations

Recommendation

- Enforce access control checks on every request
- Validate user identity and ownership
- Replace direct object references with secure indirect identifiers

5.4 Broken Authentication

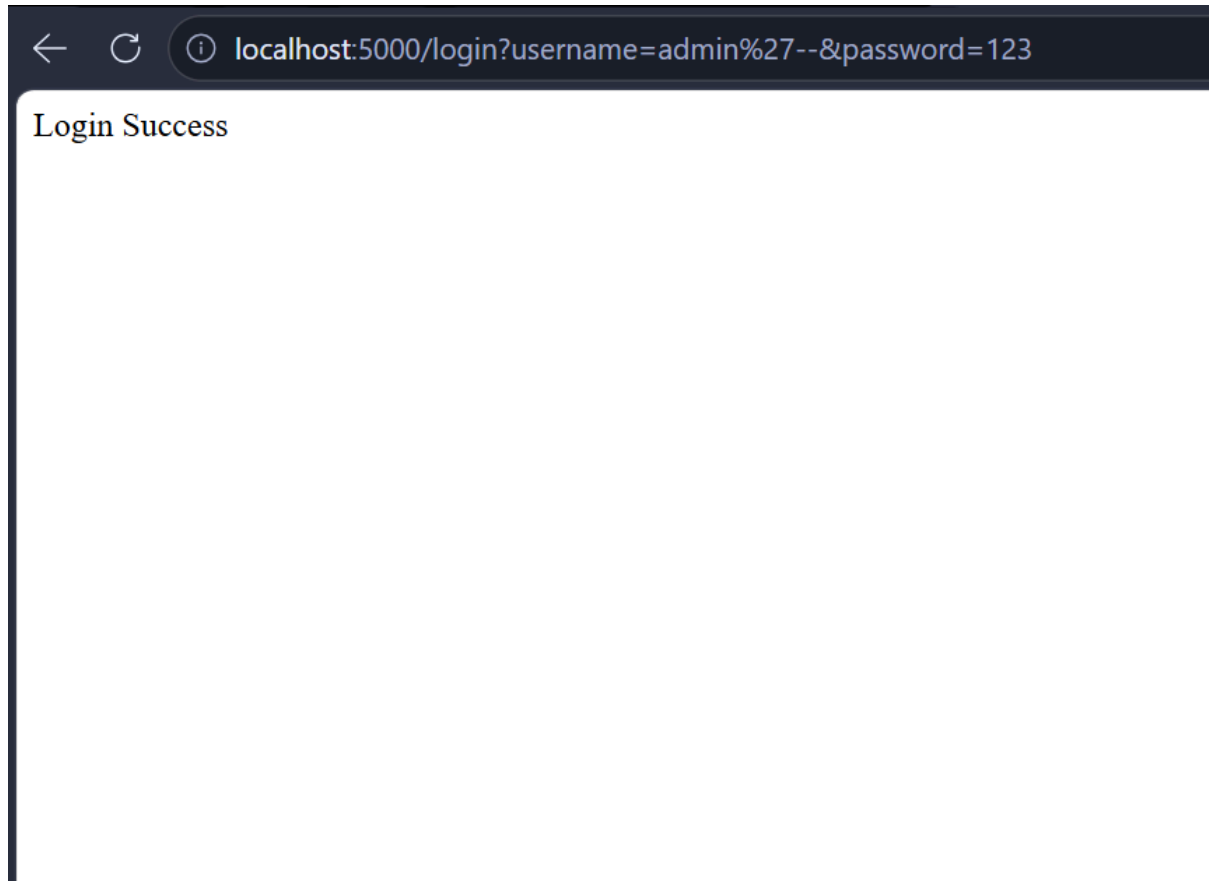
- **Severity:** High
- **CVSS Score:** 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
- **OWASP:** A07:2021 – Identification and Authentication Failures
- **CWE:** CWE-287

Description

The application lacks proper authentication enforcement and session validation mechanisms.

Impact

- Unauthorized access
- Session hijacking
- Privilege escalation



Recommendation

- Implement secure authentication mechanisms
- Use session management with expiration
- Enable secure cookies (HttpOnly, Secure)
- Add multi-factor authentication (if applicable)

5.5 Hardcoded Secrets

- **Severity:** Critical
- **CVSS Score:** 9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
- **OWASP:** A02:2021 – Cryptographic Failures

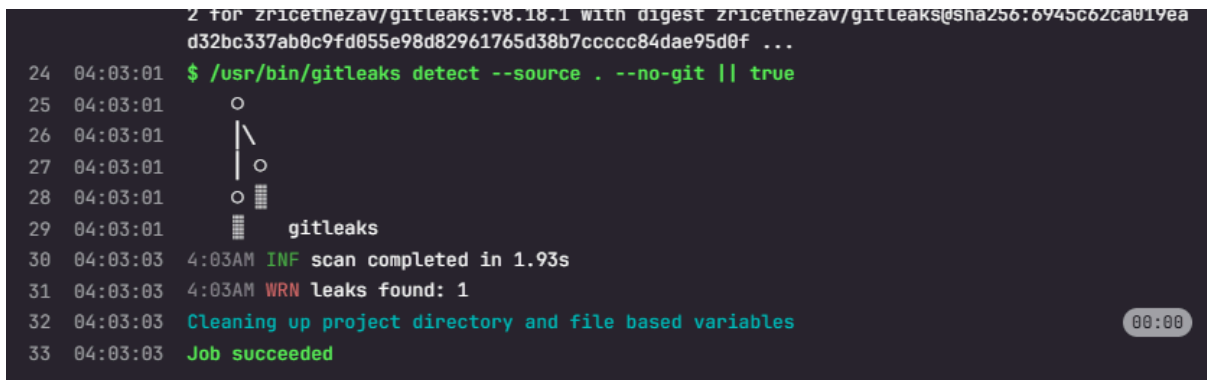
- **CWE:** CWE-798

Description

Sensitive credentials such as API keys and secret keys are hardcoded in the application source code.

Impact

- Credential leakage
- Unauthorized system access
- Potential data breaches



```

2 for zricethezav/gitleaks:v8.18.1 with digest zricethezav/gitleaks@sha256:6945c62ca019ea
d32bc337ab0c9fd055e98d82961765d38b7ccccc84dae95d0f ...
24 04:03:01 $ /usr/bin/gitleaks detect --source . --no-git || true
25 04:03:01
26 04:03:01
27 04:03:01
28 04:03:01
29 04:03:01 gitleaks
30 04:03:03 4:03AM INF scan completed in 1.93s
31 04:03:03 4:03AM WRN leaks found: 1
32 04:03:03 Cleaning up project directory and file based variables
33 04:03:03 Job succeeded
  
```

Recommendation

- Store secrets in environment variables
- Use secure secret management tools (e.g., HashiCorp Vault)
- Rotate exposed credentials immediately
- Restrict access to sensitive configuration files

5.6 Missing Security Headers

- **Severity:** Medium
- **CVSS Score:** 6.5 (AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N)
- **OWASP:** A05:2021 – Security Misconfiguration
- **CWE:** CWE-693

Description

The application does not implement essential HTTP security headers.

Findings

- Missing X-Frame-Options
- Missing Content-Security-Policy
- Missing X-Content-Type-Options

```
303 05:09:13 [*] Saving scan state, please wait...
304 05:09:13 [*] Launching module file
305 05:09:13 [*] Launching module ssl
306 05:09:13 sslscan is not installed or not found in PATH, module will be skipped
307 05:09:13 [*] Launching module cookieflags
308 05:09:13 [*] Launching module xss
309 05:09:13 [*] Launching module csp
310 05:09:13 127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
311 05:09:13 CSP is not set
312 05:09:13 [*] Launching module exec
313 05:09:13 [*] Launching module upload
314 05:09:13 [*] Launching module redirect
315 05:09:13 [*] Launching module sql
316 05:09:13 127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
317 05:09:13 [*] Launching module http_headers
318 05:09:13 127.0.0.1 - - [10/Apr/2026 05:09:13] "GET / HTTP/1.1" 200 -
319 05:09:13 Checking X-Frame-Options:
320 05:09:13 X-Frame-Options is not set
321 05:09:13 Checking X-Content-Type-Options:
322 05:09:13 X-Content-Type-Options is not set
323 05:09:13 [*] Launching module ssrf
324 05:09:16 [*] Launching module permanentxss
325 05:09:16 127.0.0.1 - - [10/Apr/2026 05:09:16] "GET / HTTP/1.1" 200 -
326 05:09:16 [*] Generating report...
327 05:09:16 A report has been generated in the file report.txt
328 05:09:18 Uploading artifacts for successful job 00:01
329 05:09:18 Uploading artifacts...
330 05:09:18 report.txt: found 1 matching artifact files and directories
331 05:09:19 Uploading artifacts as "archive" to coordinator... 201 Created correlation_id=2455125f94
e145819d3d6e9b2011bdf7 id=13862323901 responseStatus=201 Created token=eyJraWQiO
332 05:09:19 Cleaning up project directory and file based variables 00:01
333 05:09:20 Job succeeded
```

```
Gowry12 / devsecops-secure-pipeline / Jobs / #13862323892

Log timestamps in UTC. Search visible log output

628 05:05:54    python.lang.security.audit.insecure-file-permissions.insecure-file-permissions
629 05:05:54    (( Blocking ))
630 05:05:54    These permissions `${BITS}` are widely permissive and grant access to more people
        than may be
631 05:05:54    necessary. A good default is `0o644` which gives read and write access to yours
        elf and read access
632 05:05:54    to everyone else.
633 05:05:54    Details: https://sg.run/AXY4
634 05:05:54
635 05:05:54    93| os.chmod(path, (0o777 & ~current_umask() | 0o111))
636 05:05:54
637 05:05:54    app/venv/lib/python3.12/site-packages/pip/_vendor/cachecontrol/_cmd.py
638 05:05:54    python.lang.security.audit.insecure-transport.requests.request-session-with-http.r
        equest-session-with-http
639 05:05:54    (( Blocking ))
640 05:05:54    Detected a request using 'http://'. This request will be unencrypted. Use 'http
        s://' instead.
641 05:05:54    Details: https://sg.run/DoBY
642 05:05:54
643 05:05:54    33| sess.mount("http://", adapter)
644 05:05:54
645 05:05:54    app/venv/lib/python3.12/site-packages/pip/_vendor/cachecontrol/caches/file_cache.py
646 05:05:54    python.lang.security.audit.sha224-hash.sha224-hash
647 05:05:54    (( Blocking ))
648 05:05:54    This code uses a 224-bit hash function, which is deprecated or disallowed in so
        me security policies.
649 05:05:54    Consider updating to a stronger hash function such as SHA-384 or higher to ensu
        re compliance and
650 05:05:54    security.
651 05:05:54    Details: https://sg.run/Db1Yv
652 05:05:54
653 05:05:54    95| return hashlib.sha224(x.encode()).hexdigest()
654 05:05:54
655 05:05:54    app/venv/lib/python3.12/site-packages/pip/_vendor/certifi/core.py
```

Impact

- Clickjacking attacks
- Increased XSS risk
- MIME sniffing vulnerabilities

Recommendation

- Configure security headers in Flask using middleware
- Enforce CSP to restrict script execution
- Use X-Frame-Options to prevent clickjacking

5.7 Weak Cryptography

- **Severity:** Medium
- **CVSS Score:** 6.0 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N)

- **OWASP:** A02:2021 – Cryptographic Failures
- **CWE:** CWE-327

Description

The application uses weak hashing algorithms (e.g., SHA-1), which are vulnerable to collision attacks.

```

197 05:05:54      188| return Markup(value)
198 05:05:54
199 05:05:54      app/venv/lib/python3.12/site-packages/flask/sessions.py
200 05:05:54      >> python.lang.security.insecure-hash-algorithms.insecure-hash-algorithm-sha1
201 05:05:54      << Blocking >>
202 05:05:54      Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
n resistant and is
203 05:05:54      therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
204 05:05:54      Details: https://sg.run/ydYx
205 05:05:54
206 05:05:54      >> Autofix ▶ hashlib.sha256(string)
207 05:05:54      281| return hashlib.sha1(string)
208 05:05:54
209 05:05:54      app/venv/lib/python3.12/site-packages/itsdangerous-2.2.0/dist-info/METADATA
210 05:05:54      >>> generic.secrets.security.detected-jwt-token.detected-jwt-token

```

Impact

- Data integrity compromise
- Increased risk of cryptographic attacks

Recommendation

- Replace SHA-1 with secure algorithms like SHA-256 or bcrypt
- Use modern cryptographic libraries
- Implement salted hashing for sensitive data

6. DAST Scan Results

Dynamic Application Security Testing (DAST) was performed using Wapiti within the CI/CD pipeline.

Results:

- Application successfully crawled
- Multiple endpoints tested
- Vulnerabilities detected
- Report generated ([report.txt](#))

```
report.txt X
C: > Users > Administrator > AppData > Local > Temp > 4be3b172-2aa5-48c0-a9e6-393df3c3675f_artifacts.zip.75f > report
1 *****
2 Wapiti 3.2.3 - wapiti-scanner.github.io
3 Report for http://localhost:5000/
4 Date of the scan : Fri, 10 Apr 2026 05:09:13 +0000
5 Crawled pages : 1
6 Scope of the scan : folder
7 *****
8
9 Summary of vulnerabilities :
10 -----
11 Backup file : 0
12 Weak credentials : 0
13 CRLF Injection : 0
14 Content Security Policy Configuration : 1
15 Cross Site Request Forgery : 0
16 Potentially dangerous file : 0
17 Command execution : 0
18 Path Traversal : 0
19 Fingerprint web application framework : 0
20 Fingerprint web server : 0
21 Htaccess Bypass : 0
22 HTML Injection : 0
23 Clickjacking Protection : 1
24 HTTP Strict Transport Security (HSTS) : 0
25 MIME Type Confusion : 1
26 HttpOnly Flag cookie : 0
27 Unencrypted Channels : 0
28 LDAP Injection : 0
```

7. Risk Summary

Severity	Count
Critical	2
High	3
Medium	2

8. CI/CD Security Implementation

The CI/CD pipeline integrates multiple security stages:

- Build Stage
- SAST (Semgrep)
- Secret Scanning (Gitleaks)
- Dependency Analysis
- DAST (Wapiti)

This ensures continuous security validation throughout the development lifecycle.

9. Overall Recommendations

- Adopt secure coding practices
- Implement input validation and output encoding
- Remove hardcoded credentials
- Enforce strong authentication and authorization
- Configure security headers
- Integrate automated security testing into CI/CD

10. Conclusion

The assessment demonstrates the effectiveness of integrating security into the CI/CD pipeline. The identified vulnerabilities reflect common real-world threats aligned with OWASP Top 10. Addressing these issues will significantly enhance the security posture of the application.

11. Appendix

Suggested Evidence:

- SQL Injection exploitation screenshot
- XSS execution screenshot
- IDOR data access screenshot
- CI/CD pipeline execution logs
- DAST report (`report.txt`)